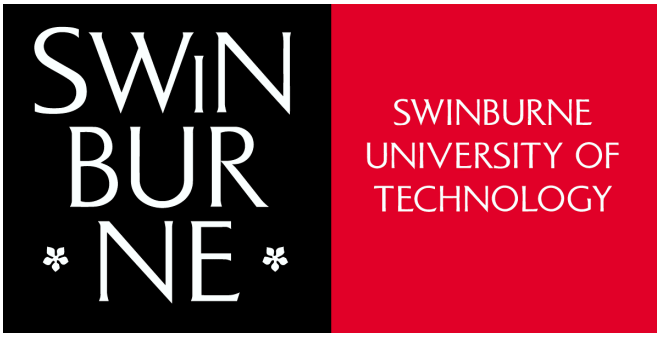
Swinburne University of Technology



COS20028 Big Data Architecture and Application – Semester 2, 2025

Code for Assignment 2

Student Name: Nguyen Pham Duy

Student ID: 104058375

Due Date: 30/07/2025

Submission Date: 30/07/2025

Part 1: Finding Corresponding Information from Multiple Sources

In this part, I have used MapReduce to get the location where the sentence “Item was defective” and where the comment with only a single word “Shoddy” appear in the data sources. Here are the scripts for the Mapper code, the Reducer code, and the Driver code.

Mapper code:

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.lib.input.FileSplit;

public class IndexMapper extends Mapper<Object, Text, Text, Text>
{

  private Text outputKey = new Text();
  private Text outputValue = new Text();
  private String filename;

  // Target comment values to match
  private static final String TARGET_1 = "Item was defective";
  private static final String TARGET_2 = "Shoddy";

  private int lineNumber = 0; // Counter to track line numbers

  @Override
  protected void setup(Context context) {
        // Get the filename of the current input split
    FileSplit fileSplit = (FileSplit) context.getInputSplit();
    filename = fileSplit.getPath().getName();
  }

  @Override
  public void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
        lineNumber++;
      String line = value.toString().trim(); // Remove
leading/trailing spaces
```

```
      if (line.isEmpty()) return;

      String[] fields = line.split("\t"); // Split the line into
tab-separated fields
      if (fields.length < 5) return; // Skip lines that don't have
at least 5 fields

      String comment = fields[4].trim(); // Get the fifth field
   // Check if comment matches one of the target patterns
      if (comment.equals(TARGET_1) ||
         (comment.equals(TARGET_2) && comment.split(" ").length ==
1)) {

          outputKey.set(comment);
          outputValue.set(filename + ": line " + lineNumber + " |
" + line); // Format the value as filename + line number +
original line
          context.write(outputKey, outputValue); // Emit key-value
pair to the reducer
      }
  }
}
```

Reducer code:

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class IndexReducer extends Reducer<Text, Text, Text, Text>
{

  @Override
  public void reduce(Text key, Iterable<Text> values, Context
context)
      throws IOException, InterruptedException {

    for (Text val : values) {
      context.write(key, val); // Emit the result
    }
```

```
  }
}
```

Driver code:

```
package stubs;
import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class InvertedIndex extends Configured implements Tool {

  public int run(String[] args) throws Exception {

    if (args.length != 2) {
      System.out.printf("Usage: InvertedIndex <input dir> <output
dir>\n");
      return -1;
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Inverted Index");
    job.setJarByClass(InvertedIndex.class);

    // Set the Map and Reduce classes
    job.setMapperClass(IndexMapper.class);
    job.setReducerClass(IndexReducer.class);

    // Set the output of the Map
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);

    // Set the output of the Reducer
    job.setOutputKeyClass(Text.class);
```

```
    job.setOutputValueClass(Text.class);

    // Set the directories of the input and the output
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    boolean success = job.waitForCompletion(true);
    return success ? 0 : 1;
  }

  public static void main(String[] args) throws Exception {
    int exitCode = ToolRunner.run(new Configuration(), new
InvertedIndex(), args);
    System.exit(exitCode);
  }
}
```

This MapReduce program identifies and indexes specific customer comments from input files. The Mapper reads each line, extracts the fifth field (comment), and checks if it exactly matches either "Item was defective" or the single-word "Shoddy". If matched, it emits the comment as the key and the filename with line number and original content as the value. The Reducer simply passes through these matched records, grouping all occurrences of each comment together. The Driver class configures and executes the job, setting the input/output paths and specifying the Mapper and Reducer classes.

Part 2: RDBMD and HiveQL

2.1. For this task, I used Pig Latin script to check if the language_code attribute in the source data is unique. Here is the code for the Pig Latin script:

```
-- Load the dataset from HDFS
data = LOAD './assignment2/austlang_dataset_nh.txt' USING
PigStorage('\t') AS (language_code:chararray,
language_name:chararray, language_synonym:chararray,
thesaurus_heading_language:chararray,
thesaurus_heading_people:chararray,
approximate_latitude_of_language_variety:chararray,
approximate_longitude_of_language_variety:chararray,
state_territory:chararray, uri:chararray);

-- Group the loaded data with language_code
grouped = GROUP data BY language_code;
```

```
-- Generate each language_code with its own count
counted = FOREACH grouped GENERATE group AS language_code,
COUNT(data) as occurrences;

-- Extract the language_code that have the count larger than 1
duplicates = FILTER counted BY occurrences > 1;

-- Store the result to the folder 'unique_language_code'
STORE duplicates INTO './assignment2/unique_language_code';
```

This Pig Latin script processes a tab-separated dataset of language records stored in HDFS. It begins by loading the data with defined schema fields. The script then groups the records by “language_code” and counts how many times each code appears. It filters out the entries that occur more than once—indicating duplicate language codes—and stores these duplicates in the “unique_language_code” output directory.

2.2 + 2.3. To prepare the data and list the unique data for all entities (tables with solid boundary) to load into MySQL and Hive, I used MapReduce to handle this. Here are the detailed scripts for processing the values in each entity.

MapReduce for “lng_id”:

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LngIdData {

    public static class LngIdMapper extends Mapper<Object, Text,
Text, Text> {
     private Text outputKey = new Text();
        private Text outputValue = new Text();

        @Override
        public void map(Object key, Text value, Context context)
```

```
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1);  //
Split the lines into fields by tab and keep empty fields
            if (fields.length > 8) { // // The length from the
beginning to the lng_st field
                // Extract the fields of lng_code, lat, lng, uri
                String lng_code = fields[0].trim();
                String lat = fields[5].trim();
                String lng = fields[6].trim();
                String uri = fields[8].trim();

                if (!lng_code.isEmpty()) {
                    outputKey.set(lng_code);
                    outputValue.set(lat + "\t" + lng + "\t" +
uri);
                    context.write(outputKey, outputValue); //
Output values with tab-separated
                }
            }
        }
    }

    public static class LngIdReducer extends Reducer<Text, Text,
Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
           for (Text val : values) {
                context.write(key, val); // / Emit final output
            }
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngIdData <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
```

```
        Job job = Job.getInstance(conf, "Language Id Data");
        job.setJarByClass(LngIdData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngIdMapper.class);
        job.setReducerClass(LngIdReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper reads each line, splits it into fields, and emits the “lng_code” as the key and a tab-separated string of “a_lng_lat”, “a_lng_lng”, and “lng_uri” as the value. The Reducer simply passes through these values, emitting each lng_code with its associated data.

MapReduce for “lng_name”:

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class LngNameData {

    public static class LngNameMapper extends Mapper<Object, Text,
Text, Text> {
        private Text name = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
            String[] fields = value.toString().split("\t"); //
Split the lines into fields by tab
            if (fields.length >= 2 && !fields[1].isEmpty()) { //
Extract the language_name field
                String[] names = fields[1].split("/"); // Split
the value in language_name by '/'
                for (String s : names) {
                    name.set(s.trim()); // Trim the value after
splitting
                    context.write(name, new Text(""));
                }
            }
        }
    }

    public static class LngNameReducer extends Reducer<Text, Text,
Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            context.write(key, null); // Emit the final output
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngNameData <input>
<output>");
            System.exit(-1);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Language Name Data");
```

```
        job.setJarByClass(LngNameData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngNameMapper.class);
        job.setReducerClass(LngNameReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper extracts the “lng_name” field from each line, splits it by the "/" delimiter to handle multiple names, trims whitespace, and emits each name as a key. The Reducer receives the unique names and outputs them as-is.

MapReduce for “lng_synonym”:

```
package stubs;

import java.io.IOException;
import java.util.HashSet;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LngSymData {
```

```
    public static class LngSymMapper extends Mapper<Object, Text,
Text, Text> {
     private Text outputKey = new Text();
        private HashSet<String> lowerSeen = new HashSet<>(); //
Tracks lowercase synonyms

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1); //
Split the lines into fields by tab and keep empty fields
            if (fields.length > 2) {
                String raw = fields[2].trim(); // Extract the
lng_synonym field
                if (!raw.isEmpty()) {
                    String[] parts = raw.split(","); // Split the
value in lng_synonym field by comma
                    for (String part : parts) {
                        String cleaned = part.trim(); // Trim
whitespace
                        if (!cleaned.isEmpty()) {
                            String lower = cleaned.toLowerCase();
// Normalize to lowercase
                            if (!lowerSeen.contains(lower)) {
                                lowerSeen.add(lower); // Add to
set to avoid duplicates
                                outputKey.set(cleaned);
                                context.write(outputKey, new
Text(""));
                            }
                        }
                    }
                } else {
                    outputKey.set(""); // Handle empty fields
                    context.write(outputKey, new Text(""));
                }
            }
        }
    }

    public static class LngSymReducer extends Reducer<Text, Text,
```

```
Text, Text> {
      private HashSet<String> lowerOutput = new HashSet<>();

        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
           String lower = key.toString().toLowerCase();
            if (!lowerOutput.contains(lower)) {
                lowerOutput.add(lower); // Ensure only one synonym
per normalized form
                context.write(key, null);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngSymData <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Language Symnonym Data");
        job.setJarByClass(LngSymData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngSymMapper.class);
        job.setReducerClass(LngSymReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper splits each “lng_synonym” string by commas, trims whitespace, normalizes to lowercase, and emits only the first occurrence of each unique synonym form to avoid case-insensitive duplicates. The Reducer further ensures that only one output is written for each normalized synonym by tracking lowercase versions and filtering duplicates.

MapReduce for “lng_thl”:

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LngThlData {

    public static class LngThlMapper extends Mapper<Object, Text,
Text, Text> {
     private Text outputKey = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1); //
Split the lines into fields by tab and keep empty fields
           if (fields.length > 3) {
                String raw = fields[3].trim(); // Extract the
lng_thl field
                if (!raw.isEmpty()) {
                    int suffixIndex = raw.lastIndexOf("language");
// Check if the string contains "language"
```

```
                    if (suffixIndex > 0) {
                        String suffix =
raw.substring(suffixIndex).trim();  // Extract the suffix
                        String[] parts = raw.substring(0,
suffixIndex).split("/"); // Split the part before "language" by /
                        for (String part : parts) {
                            String cleaned = (part.trim() + " " +
suffix).trim(); // Append the suffix
                            outputKey.set(cleaned);
                            context.write(outputKey, new
Text(""));
                        }
                    } else {
                        // No suffix found; treat whole field as
one value
                        outputKey.set(raw);
                        context.write(outputKey, new Text(""));
                    }
                } else {
                    // Include empty row
                    outputKey.set("");
                    context.write(outputKey, new Text(""));
                }
            }
        }
    }

    public static class LngThlReducer extends Reducer<Text, Text,
Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            context.write(key, null); // Emit the unique key
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngThlData <input>
<output>");
            System.exit(-1);
```

```
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Language Thl Data");
        job.setJarByClass(LngThlData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngThlMapper.class);
        job.setReducerClass(LngThlReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper splits the field by '/' if multiple entries exist, detects and appends the "language" suffix where applicable, and emits each cleaned value. If the suffix is not found, the entire field is used as-is. The Reducer ensures only unique entries are written to the output.

MapReduce for "lng_thp":

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
```

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LngThpData {

    public static class LngThpMapper extends Mapper<Object, Text,
Text, Text> {
        private Text outputKey = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1); //
Split lines into fields by tab
           if (fields.length > 4) {
                String raw = fields[4].trim(); // // Extract the
lng_thp field
                if (!raw.isEmpty()) {
                    int suffixIndex = raw.lastIndexOf("people");
// Check if the string contains "people"
                    if (suffixIndex > 0) {
                        String suffix =
raw.substring(suffixIndex).trim();  // Extract the suffix
                        String[] parts = raw.substring(0,
suffixIndex).split("/"); // Split the part before "people" by /
                        for (String part : parts) {
                            String cleaned = (part.trim() + " " +
suffix).trim(); // Append the suffix
                            outputKey.set(cleaned);
                            context.write(outputKey, new
Text(""));
                        }
                    } else {
                        // No suffix found; treat whole field as
one value
                        outputKey.set(raw);
                        context.write(outputKey, new Text(""));
                    }
                } else {
                    // Include empty row
                    outputKey.set("");
```

```
                    context.write(outputKey, new Text(""));
                }
            }
        }
    }

    public static class LngThpReducer extends Reducer<Text, Text,
Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            context.write(key, null); // Emit the unique key
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngThpData <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Language Thp Data");
        job.setJarByClass(LngThpData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngThpMapper.class);
        job.setReducerClass(LngThpReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

```
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper splits the field by '/' if multiple entries exist, detects and appends the "people" suffix where applicable, and emits each cleaned value. If the suffix is not found, the entire field is used as-is. The Reducer ensures only unique entries are written to the output.

MapReduce for "lng_st":

```
package stubs;

import java.io.IOException;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LngStData {

    public static class LngStMapper extends Mapper<Object, Text,
Text, Text> {
     private Text outputKey = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1); //
Split the lines into fields by tab and keep empty fields
            // The length from the beginning to the lng_st field
           if (fields.length > 7) {
                String raw = fields[7].trim(); // lng_st field
                if (!raw.isEmpty()) {
                    String[] parts = raw.split(",");  // Split the
value in lng_st field by comma
                    for (String part : parts) {
```

```
                        String cleaned = part.trim(); // Trim the
value after splitting
                        if (!cleaned.isEmpty()) {
                            outputKey.set(cleaned);
                            context.write(outputKey, new
Text("")); // Output values
                        }
                    }
                } else {
                    // Include empty entry
                    outputKey.set("");
                    context.write(outputKey, new Text(""));
                }
            }
        }
    }

    public static class LngStReducer extends Reducer<Text, Text,
Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            context.write(key, null); // Emit final output with
states
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: LngStData <input>
<output>");
            System.exit(-1);
        }
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "LngStData Data");
        job.setJarByClass(LngStData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(LngStMapper.class);
        job.setReducerClass(LngStReducer.class);
```

```
        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper processes each input line, splits the “lng_st” field by commas, trims each part, and emits individual state or territory names. If the field is empty, it emits a blank entry. The Reducer outputs each unique state or territory, effectively deduplicating the list.

(*) Besides, there is a question, “How many counts does QLD have?”. Here is the MapReduce to find the answer:

Mapper code:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class StateMapper extends Mapper<Object, Text, Text,
IntWritable> {

  private Text state = new Text();

  @Override
  public void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
        String[] fields = value.toString().split("\t"); // Split
```

```
the lines into fields
        if (fields.length >= 9) {
              String[] states = fields[7].split(","); // Extract
the lng_state fields and split by comma
              for (String s : states) {
                    state.set(s);
                    context.write(state, new IntWritable(1)); //
Emit states with count 1
              }
        }
  }
}
```

Reducer code:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class CountReducer extends Reducer<Text, IntWritable, Text,
IntWritable> {

  @Override
  public void reduce(Text key, Iterable<IntWritable> values,
Context context)
      throws IOException, InterruptedException {

        int count = 0;
     // Loop through all values associated with the key and sum
them
      for (IntWritable val : values) {
          count += val.get(); // Add each value to the running
total
      }
      context.write(key, new IntWritable(count)); // Emit state
and its total count
  }
}
```

Driver code:

```
package stubs;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class UniqueStateDriver extends Configured implements Tool
{

  public int run(String[] args) throws Exception {

    if (args.length != 2) {
      System.out.printf("Usage: Unique State Count <input dir>
<output dir>\n");
      return -1;
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "UniqueSateCount");
    job.setJarByClass(UniqueStateDriver.class);

    // Set the Map and Reduce classes
    job.setMapperClass(StateMapper.class);
    job.setReducerClass(CountReducer.class);

    // Set the output of the Map
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(IntWritable.class);

    // Set the output of the Reducer
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    // Set the directories of the input and the output
    FileInputFormat.addInputPath(job, new Path(args[0]));
```

```
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    boolean success = job.waitForCompletion(true);
    return success ? 0 : 1;
  }

  public static void main(String[] args) throws Exception {
    int exitCode = ToolRunner.run(new Configuration(), new
UniqueStateDriver(), args);
    System.exit(exitCode);
  }
}
```

The Mapper reads each line of the dataset, extracts the “lng_st” information from the eighth field (split by commas), and emits each state with a count of 1. The Reducer then sums these counts for each unique state and outputs the total number of records associated with that state.

2.4. To prepare the data for all weak entities (tables with the dashed boundary) to load into MySQL and Hive, I continued using MapReduce to solve this task. Here are the detailed scripts for processing the values in each weak entity.

MapReduce for “rel_code_name”:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class RelCodeNameData {

    public static class RelCodeNameMapper extends Mapper<Object,
Text, Text, Text> {
        private Text output = new Text();
```

```
        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
            String[] fields = value.toString().split("\t"); //
Split the lines to fields by tab
            String code = fields[0].trim(); // Get and trim the
value of the language_code
            if (fields.length >= 2 && !code.isEmpty()) {
                String namesField = fields[1].trim(); // Get and
trim the language_name
                if (!namesField.isEmpty()) {
                String[] names = namesField.split("/"); // Split
the value in language_name field
                for (String name : names) {
                      output.set(code + "\t" + name.trim()); //
Set the output with code and name
                      context.write(output, new Text(""));
                }
                } else {
                output.set(code + "\t"); // Emit when no value in
name
                context.write(output, new Text(""));
                }
            }
        }
    }

    public static class RelCodeNameReducer extends Reducer<Text,
Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            context.write(key, null); // Emit the key
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: RelCodeNameData <input>
<output>");
            System.exit(-1);
```

```
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "RelCodeNameData Data");
        job.setJarByClass(RelCodeNameData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(RelCodeNameMapper.class);
        job.setReducerClass(RelCodeNameReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper splits each input line, extracts the “lng_code” and splits the “lng_name” field by ‘/’ if multiple names exist, then emits each pair as a tab-separated key. The Reducer outputs each unique (lng_code lng_name) combination.

MapReduce for “rel_code_synonym”:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
```

```
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class RelCodeSymData {

    public static class RelCodeSymMapper extends Mapper<Object,
Text, Text, Text> {
     private Text outputKey = new Text();
        private Text outputValue = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1); //
Split the lines into fields by tab and keep empty fields
            if (fields.length > 2) {
                String lng_code = fields[0].trim(); // Get and
trim the value of the lng_code
                String raw = fields[2].trim(); // Extract and trim
the lng_synonym field
                if (!lng_code.isEmpty()) {
                    if (!raw.isEmpty()) {
                        String[] parts = raw.split(","); // Split
the value in lng_synonym by comma
                        for (String part : parts) {
                            String cleaned = part.trim();
                            if (!cleaned.isEmpty()) {
                                outputKey.set(lng_code);
                                outputValue.set(cleaned);
                                context.write(outputKey,
outputValue); // Emit both code and synonym
                            }
                        }
                    } else {
                        outputKey.set(lng_code);
                        outputValue.set("");
                        context.write(outputKey, outputValue); //
Emit code with blank
                    }
                }
            }
```

```
        }
    }

    public static class RelCodeSymReducer extends Reducer<Text,
Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
           for (Text val : values) {
                context.write(key, val); // Emit the result
            }
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: RelCodeSymData <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "RelCodeSym Data");
        job.setJarByClass(RelCodeSymData.class);
        // Set the Map and Reduce classes
        job.setMapperClass(RelCodeSymMapper.class);
        job.setReducerClass(RelCodeSymReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
```

```
    }
}
```

The Mapper splits the “lng_synonym” string by commas, trims each value, and emits each synonym alongside its corresponding “lng_code”. If the synonym field is empty, it still emits the code with a blank value. The Reducer outputs all (lng_code lng_synonym) pairs, preserving multiple synonyms per code.

MapReduce for “rel_code_thl”:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class RelCodeThlData {

    public static class RelCodeThlMapper extends Mapper<Object,
Text, Text, Text> {
     private Text outputKey = new Text();
        private Text outputValue = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
            String[] fields = value.toString().split("\t", -1);
// Split the lines into fields by tab and keep empty fields
             if (fields.length > 3) {
                 String lng_code = fields[0].trim(); // Get the
value lng_code
                 String raw = fields[3].trim(); // Get the value
of lng_thl
                 if (!lng_code.isEmpty()) {
                     if (!raw.isEmpty()) {
```

```
                        int suffixIndex =
raw.lastIndexOf("language"); // Look for the "language" suffix
                        if (suffixIndex > 0) {
                            String suffix =
raw.substring(suffixIndex).trim(); // Extract the suffix
                            String[] parts = raw.substring(0,
suffixIndex).split("/"); // Split the pre-suffix by '/'
                            for (String part : parts) {
                                String cleaned = (part.trim() + "
" + suffix).trim();
                                outputKey.set(lng_code);
                                outputValue.set(cleaned);
                                context.write(outputKey,
outputValue); // Emit the code with normalized lng_thl
                            }
                        } else {
                            outputKey.set(lng_code);
                            outputValue.set(raw);
                            context.write(outputKey,
outputValue); // Emit code with raw when no suffix found
                        }
                    } else {
                        // Empty lng_thl, preserve lng_code with
empty string
                        outputKey.set(lng_code);
                        outputValue.set("");
                        context.write(outputKey, outputValue);
                    }
                }
            }
        }

    public static class RelCodeThlReducer extends Reducer<Text,
Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            for (Text val : values) {
                context.write(key, val); // Emit the result
            }
```

```
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: RelCodeThl <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "RelCodeThl Data");
        job.setJarByClass(RelCodeThlData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(RelCodeThlMapper.class);
        job.setReducerClass(RelCodeThlReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper parses the “lng_thl” field, detects and preserves the "language" suffix if present, splits multiple values by ‘/’, and emits cleaned (lng_code lng_thl) pairs. If the field is empty or lacks the suffix, it still outputs the original values. The Reducer passes these pairs through for final output.

MapReduce for “rel_code_thp”:

```
package stubs;
```

```
import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class RelCodeThpData {

    public static class RelCodeThpMapper extends Mapper<Object,
Text, Text, Text> {
     private Text outputKey = new Text();
        private Text outputValue = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
           String[] fields = value.toString().split("\t", -1);  //
Split the lines into fields by tab and keep empty fields
            if (fields.length > 4) {
                String lng_code = fields[0].trim(); // Get the
value lng_code
                String raw = fields[4].trim(); // Get the value of
lng_thp
                if (!lng_code.isEmpty()) {
                    if (!raw.isEmpty()) {
                        int suffixIndex =
raw.lastIndexOf("people"); // Look for the "people" suffix
                        if (suffixIndex > 0) {
                            String suffix =
raw.substring(suffixIndex).trim(); // Extract the suffix
                            String[] parts = raw.substring(0,
suffixIndex).split("/");  // Split the pre-suffix by '/'
                            for (String part : parts) {
                                String cleaned = (part.trim() + "
" + suffix).trim();
                                outputKey.set(lng_code);
```

```
                                outputValue.set(cleaned);
                                context.write(outputKey,
outputValue); // Emit the code with normalized lng_thp
                            }
                        } else {
                            outputKey.set(lng_code);
                            outputValue.set(raw);
                            context.write(outputKey, outputValue);
                        }
                    } else {
                        // Empty lng_thp, preserve lng_code with
empty string
                        outputKey.set(lng_code);
                        outputValue.set("");
                        context.write(outputKey, outputValue);
                    }
                }
            }
        }
    }

    public static class RelCodeThpReducer extends Reducer<Text,
Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
            for (Text val : values) {
                context.write(key, val);
            }
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: RelCodeThp <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "RelCodeThp Data");
```

```
        job.setJarByClass(RelCodeThpData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(RelCodeThpMapper.class);
        job.setReducerClass(RelCodeThpReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper parses the “lng_thp” field, detects and preserves the "people" suffix if present, splits multiple values by ‘/’, and emits cleaned (lng_code lng_thp) pairs. If the field is empty or lacks the suffix, it still outputs the original values. The Reducer passes these pairs through for final output.

MapReduce for “rel_code_st”:

```
package stubs;

import java.io.IOException;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
public class RelCodeStData {

    public static class RelCodeStMapper extends Mapper<Object,
Text, Text, Text> {
     private Text outputKey = new Text();
        private Text outputValue = new Text();

        @Override
        public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {
            String[] fields = value.toString().split("\t", -1); //
Split the lines into fields by tab and keep empty fields
             if (fields.length > 7) {
                 String lng_code = fields[0].trim(); // Get and
trim the value of the lng_code
                 String raw = fields[7].trim(); // Extract and
trim the lng_st field
                 if (!lng_code.isEmpty()) {
                     if (!raw.isEmpty()) {
                         String[] parts = raw.split(","); // Split
the value in lng_st by comma
                         for (String part : parts) {
                             String cleaned = part.trim(); // Trim
the value
                             outputKey.set(lng_code);
                             outputValue.set(cleaned);
                             context.write(outputKey,
outputValue); // Emit both code and state
                         }
                     } else {
                         outputKey.set(lng_code);
                         outputValue.set("");
                         context.write(outputKey, outputValue); //
Emit code with blank
                     }
                 }
             }
        }
    }

    public static class RelCodeStReducer extends Reducer<Text,
Text, Text, Text> {
```

```
        @Override
        public void reduce(Text key, Iterable<Text> values,
Context context)
                throws IOException, InterruptedException {
           for (Text val : values) {
                context.write(key, val); // Emit the result
            }
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.out.println("Usage: RelCodeStData <input>
<output>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "RelCodeSt Data");
        job.setJarByClass(RelCodeStData.class);

        // Set the Map and Reduce classes
        job.setMapperClass(RelCodeStMapper.class);
        job.setReducerClass(RelCodeStReducer.class);

        // Set the output of the Map
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Set the output of the Reducer
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        //Set the directories of the input and the output
        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

The Mapper extracts the “lng_st” field, splits multiple entries by commas, trims whitespace, and emits each (lng_code lng_st) pair. If the field is empty, it still

outputs the code with a blank value. The Reducer passes through all the pairs for output.

2.5. Here are two scripts to create tables and import data into the created tables for both MySQL and Hive. The database “indigenous” is also created and used.
MySQL script:

```
-- Create and use database "indigenous":
CREATE DATABASE IF NOT EXISTS indigenous;
USE indigenous;

-- Create and import data into "lng_id" table:
CREATE TABLE lng_id (lng_code VARCHAR(10) PRIMARY KEY, a_lng_lat
VARCHAR(50), a_lng_lng VARCHAR(50), lng_uri VARCHAR(255));
LOAD DATA INFILE '/home/training/asm2/lng_id_data/part-r-00000'
INTO TABLE lng_id FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';

-- Create and import data into "lng_name" table:
CREATE TABLE lng_name (lng_name VARCHAR(255) PRIMARY KEY);
LOAD DATA INFILE '/home/training/asm2/lng_name_data/part-r-00000'
INTO TABLE lng_name FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';

-- Create and import data into "rel_code_name" table:
CREATE TABLE rel_code_name (lng_code VARCHAR(10), lng_name
VARCHAR(255), IDKey INT PRIMARY KEY AUTO_INCREMENT, FOREIGN KEY
(lng_code) REFERENCES lng_id(lng_code), FOREIGN KEY (lng_name)
REFERENCES lng_name(lng_name));
LOAD DATA INFILE
'/home/training/asm2/rel_code_name_data/part-r-00000' INTO TABLE
rel_code_name FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';

-- Create and import data into "lng_synonym" table:
CREATE TABLE lng_synonym (lng_synonym VARCHAR(255) PRIMARY KEY);
LOAD DATA INFILE
'/home/training/asm2/lng_synonym_data/part-r-00000' INTO TABLE
lng_synonym FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';


-- Create and import data into "rel_code_synonym" table:
CREATE TABLE rel_code_synonym (lng_code VARCHAR(10), lng_synonym
VARCHAR(255), IDKey INT PRIMARY KEY AUTO_INCREMENT, FOREIGN KEY
```

```
(lng_code) REFERENCES lng_id(lng_code), FOREIGN KEY (lng_synonym)
REFERENCES lng_synonym(lng_synonym));
LOAD DATA INFILE
'/home/training/asm2/rel_code_synonym_data/part-r-00000' INTO
TABLE rel_code_synonym FIELDS TERMINATED BY '\t' LINES TERMINATED
BY '\n';

-- Create and import data into "lng_thl" table:
CREATE TABLE lng_thl (lng_thl VARCHAR(255) PRIMARY KEY);
LOAD DATA INFILE '/home/training/asm2/lng_thl_data/part-r-00000'
INTO TABLE lng_thl FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';

-- Create and import data into "rel_code_thl" table:
CREATE TABLE rel_code_thl (lng_code VARCHAR(10), lng_thl
VARCHAR(255), IDKey INT PRIMARY KEY AUTO_INCREMENT, FOREIGN KEY
(lng_code) REFERENCES lng_id(lng_code), FOREIGN KEY (lng_thl)
REFERENCES lng_thl(lng_thl));
LOAD DATA INFILE
'/home/training/asm2/rel_code_thl_data/part-r-00000' INTO TABLE
rel_code_thl FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';

-- Create and import data into "lng_thp" table:
CREATE TABLE lng_thp (lng_thp VARCHAR(255) PRIMARY KEY);
LOAD DATA INFILE '/home/training/asm2/lng_thp_data/part-r-00000'
INTO TABLE lng_thp FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';

-- Create and import data into "rel_code_thp" table:
CREATE TABLE rel_code_thp (lng_code VARCHAR(10), lng_thp
VARCHAR(255), IDKey INT PRIMARY KEY AUTO_INCREMENT, FOREIGN KEY
(lng_code) REFERENCES lng_id(lng_code), FOREIGN KEY (lng_thp)
REFERENCES lng_thp(lng_thp));
LOAD DATA INFILE
'/home/training/asm2/rel_code_thp_data/part-r-00000' INTO TABLE
rel_code_thp FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';

-- Create and import data into "lng_st" table:
CREATE TABLE lng_st (lng_st VARCHAR(50) PRIMARY KEY);
LOAD DATA INFILE '/home/training/asm2/lng_st_data/part-r-00000'
INTO TABLE lng_st FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';
```

```
-- Create and import data into "rel_code_st" table:
CREATE TABLE rel_code_st (lng_code VARCHAR(10), lng_st
VARCHAR(50), IDKey INT PRIMARY KEY AUTO_INCREMENT, FOREIGN KEY
(lng_code) REFERENCES lng_id(lng_code), FOREIGN KEY (lng_st)
REFERENCES lng_st(lng_st));
LOAD DATA INFILE
'/home/training/asm2/rel_code_st_data/part-r-00000' INTO TABLE
rel_code_st FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';
```

Hive script:

```
-- Create and use database "indigenous":
CREATE DATABASE IF NOT EXISTS indigenous;
USE indigenous;

-- Create and import data into "lng_id" table:
CREATE TABLE lng_id (lng_code STRING, a_lng_lat STRING, a_lng_lng
STRING, lng_uri STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY
'\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_id_data/part-r-00000' INTO TABLE
lng_id;

-- Create and import data into "lng_name" table:
CREATE TABLE lng_name (lng_name STRING) ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_name_data/part-r-00000' INTO
TABLE lng_name;

-- Create and import data into "rel_code_name" table:
CREATE TABLE rel_code_name (lng_code STRING, lng_name STRING) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';
LOAD DATA INPATH 'assignment2/rel_code_name_data/part-r-00000'
INTO TABLE rel_code_name;

-- Create and import data into "lng_synonym" table:
CREATE TABLE lng_synonym (lng_synonym STRING) ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_synonym_data/part-r-00000' INTO
TABLE lng_synonym;
```

```
-- Create and import data into "rel_code_synonym" table:
CREATE TABLE rel_code_synonym (lng_code STRING, lng_synonym
STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' LINES
TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/rel_code_synonym_data/part-r-00000'
INTO TABLE rel_code_synonym;

-- Create and import data into "lng_thl" table:
CREATE TABLE lng_thl (lng_thl STRING) ROW FORMAT DELIMITED FIELDS
TERMINATED BY '\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_thl_data/part-r-00000' INTO
TABLE lng_thl;

-- Create and import data into "rel_code_thl" table:
CREATE TABLE rel_code_thl (lng_code STRING, lng_thl STRING) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';
LOAD DATA INPATH 'assignment2/rel_code_thl_data/part-r-00000' INTO
TABLE rel_code_thl;

-- Create and import data into "lng_thp" table:
CREATE TABLE lng_thp (lng_thp STRING) ROW FORMAT DELIMITED FIELDS
TERMINATED BY '\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_thp_data/part-r-00000' INTO
TABLE lng_thp;

-- Create and import data into "rel_code_thp" table:
CREATE TABLE rel_code_thp (lng_code STRING, lng_thp STRING) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t' LINES TERMINATED BY
'\n';
LOAD DATA INPATH 'assignment2/rel_code_thp_data/part-r-00000' INTO
TABLE rel_code_thp;

-- Create and import data into "lng_st" table:
CREATE TABLE lng_st (lng_st STRING) ROW FORMAT DELIMITED FIELDS
TERMINATED BY '\t' LINES TERMINATED BY '\n';
LOAD DATA INPATH 'assignment2/lng_st_data/part-r-00000' INTO TABLE
lng_st;

-- Create and import data into "rel_code_st" table:
CREATE TABLE rel_code_st (lng_code STRING, lng_st STRING) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t' LINES TERMINATED BY
```

```
'\n';
LOAD DATA INPATH 'assignment2/rel_code_st_data/part-r-00000' INTO
TABLE rel_code_st;
```

2.6. To show the detailed information of table “rel_code_st” in MySQL, I used this SQL code:

```
DESCRIBE rel_code_st;
```

2.7. Here is the SQL code to retrieve the results containing “lng_code”, “lng_name”, and “lng_st” of tuples whose lng_name starts with the upper case “D” for both MySQL and Hive:

```
SELECT l.lng_code, n.lng_name, s.lng_st
FROM lng_id l
JOIN rel_code_name rcn ON l.lng_code = rcn.lng_code
JOIN lng_name n ON rcn.lng_name = n.lng_name
JOIN rel_code_st rcs ON l.lng_code = rcs.lng_code
JOIN lng_st s ON rcs.lng_st = s.lng_st
WHERE n.lng_name LIKE 'D%';
```

To collect the result, the SQL query needs to retrieve language records from a relational database by joining multiple related tables. It starts with the “lng_id” table (aliased as l) and joins it with the “rel_code_name” and “lng_name” tables to access the language names. It also joins with “rel_code_st” and “lng_st” to retrieve associated state information. The query filters the results to include only those where the lng_name starts with the uppercase letter “D”. The final output includes the language code (lng_code), the name of the language (lng_name), and the state (lng_st) associated with each matching entry.

2.8. Here are the SQL codes to retrieve the results containing “lng_code”, “lng_name”, “lng_st”, “a_lng_lat”, and “a_lng_lng” of tuples whose “lng_synonym” contains “Kerama” or contains exactly “Kerama” for both MySQL and Hive:
SQL code for containing exactly “Kerama” in “lng_synonym”:

```
SELECT l.lng_code, n.lng_name, s.lng_st, l.a_lng_lat, l.a_lng_lng
FROM lng_id l
JOIN rel_code_synonym rcs ON l.lng_code = rcs.lng_code
JOIN lng_synonym syn ON rcs.lng_synonym = syn.lng_synonym
JOIN rel_code_name rcn ON l.lng_code = rcn.lng_code
JOIN lng_name n ON rcn.lng_name = n.lng_name
JOIN rel_code_st r ON l.lng_code = r.lng_code
JOIN lng_st s ON r.lng_st = s.lng_st
WHERE syn.lng_synonym = 'Kerama';
```

SQL code for containing “Kerama” in “lng_synonym”:

```
SELECT l.lng_code, n.lng_name, s.lng_st, l.a_lng_lat, l.a_lng_lng
FROM lng_id l
JOIN rel_code_synonym rcs ON l.lng_code = rcs.lng_code
JOIN lng_synonym syn ON rcs.lng_synonym = syn.lng_synonym
JOIN rel_code_name rcn ON l.lng_code = rcn.lng_code
JOIN lng_name n ON rcn.lng_name = n.lng_name
JOIN rel_code_st r ON l.lng_code = r.lng_code
JOIN lng_st s ON r.lng_st = s.lng_st
WHERE syn.lng_synonym LIKE '%Kerama%';
```

To collect the result, the SQL query retrieves detailed information about languages whose synonym contains “Kerama”, which has two cases. The first case contains the exact term, and the second case contains a synonym that contains the term. It begins with the “lng_id” table (aliased as l) to access core language data, including geographic coordinates (a_lng_lat and a_lng_lng). The query joins “rel_code_synonym” and “lng_synonym” to filter for entries containing exactly “Kerama” with “syn.lng_synonym = 'Kerama'” or containing “Kerama” with “syn.lng_synonym LIKE '%Kerama%'”. It also joins “rel_code_name” and “lng_name” to retrieve the full language name, and “rel_code_st” and “lng_st” to obtain the state in which the language is associated. The final output includes the language code, name, state, latitude, and longitude for all matching records.